University of Exeter

ECM2433: The C Family

Program structure and compilation

Stack program

```c
#include <stdio.h>
#include <stdlib.h>

/*----- structure definition -----------------*/
struct stackStruct
{
    char                 data;
    struct stackStruct   *next;
};
typedef struct stackStruct STACK;

/*----- function prototypes ------------------*/
extern STACK  *push(STACK *, char);
extern char   pop(STACK **);
extern void   printStack(STACK *);
extern void   emptyStack(STACK **);

/*----- main function ------------------------*/
int main ()
{
    STACK   *stack = NULL;

    printStack(stack);
    stack = push(stack,'Q');
    stack = push(stack,'C');
    printStack(stack);
    stack = push(stack,'A');

    char c = pop(&stack);
    printStack(stack);

    emptyStack(&stack);
    return 0;
}
```

#includes

structure for a node

function prototypes

main function

other functions

Start of a C program implementing and using a stack as a linked list.

Stack program: restructuring

```c
#include <stdio.h>
#include <stdlib.h>

/*----- structure definition -----------------*/
struct stackStruct
{
    char                 data;
    struct stackStruct   *next;
};
typedef struct stackStruct STACK;

/*----- function prototypes ------------------*/
extern STACK  *push(STACK *, char);
extern char   pop(STACK **);
extern void   printStack(STACK *);
extern void   emptyStack(STACK **);

/*----- main function ------------------------*/
int main ()
{
    STACK   *stack = NULL;

    printStack(stack);
    stack = push(stack,'Q');
    stack = push(stack,'C');
    printStack(stack);
    stack = push(stack,'A');

    char c = pop(&stack);
    printStack(stack);

    emptyStack(&stack);
    return 0;
}
```

All this code moved into a file called "`stack.h`"

Defines the API

Stack program: restructuring

University of Exeter

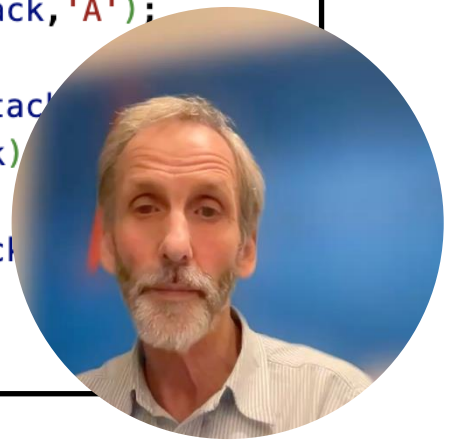
```c
#include <stdio.h>
#include <stdlib.h>

/*----- structure definition -----------------*/
struct stackStruct
{
    char                 data;
    struct stackStruct   *next;
};
typedef struct stackStruct STACK;

/*----- function prototypes ------------------*/
extern STACK  *push(STACK *, char);
extern char   pop(STACK **);
extern void   printStack(STACK *);
extern void   emptyStack(STACK **);

/*----- main function ------------------------*/
int main ()
{
    STACK   *stack = NULL;

    printStack(stack);
    stack = push(stack,'Q');
    stack = push(stack,'C');
    printStack(stack);
    stack = push(stack,'A');

    char c = pop(&stack);
    printStack(stack);

    emptyStack(&stack);
    return 0;
}
```

All this code moved into a file called "stack.h"

Defines the API

Main program in e.g. "myProgram.c"

```c
#include <stdio.h>
#include <stdlib.h>

#include "stack.h"

int main ()
{
    STACK   *stack = NULL;

    printStack(stack);
    stack = push(stack,'Q');
    stack = push(stack,'C');
    printStack(stack);
    stack = push(stack,'A');

    char c = pop(&stac
    printStack(stack)

    emptyStack(&stac
    return 0;
}
```



Stack implementation of API in "stack.c"

Stack program: restructuring

```c
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>

#include "stack.h"  // Def'n of STACK and prototypes

STACK *newNode (char data)
{
    // Create a new node holding the given data and return a pointer to it.
   STACK *s = (STACK *)malloc(sizeof(STACK));
   if (s==NULL) {
      fprintf(stderr, "Memory allocation error (%s)\n", strerror(errno));
      exit(-1);
   }
   s->data = data;
   s->next = NULL;
   return s;
}

char pop(STACK **stack)
{
    // Pop a value of the stack
```

Stack implementation of API in "stack.c"

Shared code: the stack utility

University of Exeter

- **stack.h**
  - #include-s any .h files needed by stack.c
  - defines structures, typedefs etc
  - defines function prototypes (push, pop, printStack, emptyStack)
  - is #include-ed into any program that wants to use the stack functionality
- **stack.c**
  - #include-s stack.h
  - fully defines the functions
  - **does not contain a main function**
  - is compiled into stack.o, **but not linked**
- **stack.o**
  - is eventually linked to the program that wants to use the functionality

Shared code: the stack utility

stack.c will compile, but not link:

```
$ gcc stack.c
/usr/bin/ld: /usr/lib/gcc/x86_64-redhat-linux/4.4.7/../../../../
                              lib64/crt1.o: In function `_start':
(.text+0x20): undefined reference to `main'
collect2: ld returned 1 exit status
```

because it has no main function

Shared code: storage classes

static: The newNode function is only available to functions within **stack.c**

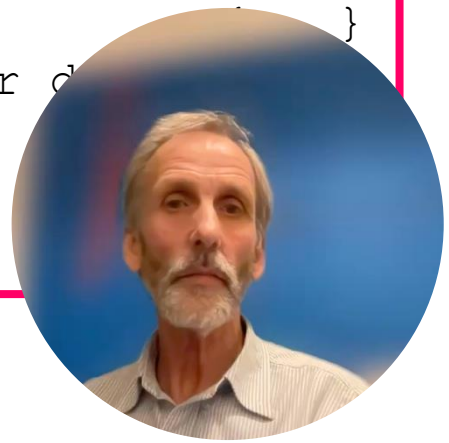
extern: The functions are available to all functions within **stack.c** and **myProgram.c**

stack.h

```
/* function prototypes */
static STACK *newNode     (char);
extern STACK  push        (STACK *, char);
extern char   pop         (STACK **);
extern void   printStack  (STACK *);
extern void   emptyStack  (STACK **);
```

stack.c

```
/* function definitions */
static STACK *newNode     (char data)                    }
extern STACK  push        (STACK *stack, char d
extern char   pop         (STACK **stack)
extern void   printStack  (STACK *stack)
extern void   emptyStack  (STACK **stack)
```



Storage classes

University of Exeter

auto (default)

```
int i;
```

- declared at the start of a block
- storage allocated when block entered and automatically freed when block exited

register

```
register int i;
```

- variable is stored in CPU registers (rather than memory) for quick access

static

```
static int pageNum;
```

- variable continues to exist even after the block in which it is defined terminates
- value is retained between repeated calls to the same function
- scope is limited to the block in which it is defined
- function is only visible within its “translation unit” (i.e. its .c file)

extern

```
extern int
```

- scope is global
- function is visible globally

Multiple includes

Consider a program structure like this:

grandparent.h

```
struct dataStruct
{ char    x; };
```

parent.h

```
#include <grandparent.h>
```

child.c

```
#include <parent.h>
#include <grandparent.h>

int main ()
{ return 0; }
```

Multiple includes: compile

```
$ gcc -c child.c -o child.o

In file included from child.c:2:0:
./grandparent.h:1:8: error: redefinition of   struct dataStruct
struct dataStruct
       ^
In file included from ./parent.h:1:0,
                 from child.c:1:
./grandparent.h:1:8: note: originally defined here
struct dataStruct
       ^
```

The structure has been defined twice.

#include guards

To avoid trying to include the same code multiple times in one executable, each header file needs a #include guard:

```
#ifndef __STACK_H
#define __STACK_H

/* content of stack header file, stack.h */

#endif
```

Must be unique: standard is double-underscore, followed by file name in capitals, with dots replaced by underscores.

Compile and Link with header files

To compile the two program modules (separately):

```
$ gcc  -c stack.c -o stack.o
$ gcc  -c myProgram.c -o myProgram.o
```

and then link them together:

```
$ gcc myProgram.o stack.o -o myProgram
```

Compile and Link with header files

To compile the two program modules (separately):

```
$ gcc  -c stack.c -o stack.o
$ gcc  -c myProgram.c -o myProgram.o
```

and then link them together:

```
$ gcc myProgram.o stack.o -o myProgram
```

```
$ gcc  -c another.c -o another.o    (another.c #includes ”stack.h”)
$ gcc another.o stack.o -o another
```

My maths program

```
#include <stdio.h>
#include <math.h>

int main (void)
{
   float radians = 134.5*M_PI/180;
   printf("cos(%f) = %f\n", radians, cosf(radians));
   return 0;
}
```

My maths program

Try compiling this program:

```
$ gcc -c myMaths.c -o myMaths.o
```

and linking it:

```
$ gcc myMaths.o -o myMaths
```

Result:

```
myMaths.o: In function `main':
myMaths.c:(.text+0x19): undefined reference to `cosf'
collect2: ld returned 1 exit status
```

Linking and libraries

Libraries group together multiple object files (*.o) into a single unit.

Static libraries (.a)

- Linked with, and becomes part of, the application at link time.

Dynamically linked shared object libraries (.so)

- Linked to at runtime.

```
$ find /usr -name "libm.*"
/usr/lib/x86_64-redhat-linux5E/lib64/libm.so
/usr/lib/x86_64-redhat-linux5E/lib64/libm.a
/usr/lib64/libm.so
/usr/lib64/libm.a
```

My maths program

Try compiling this program:

```
$ gcc -c myMaths.c -o myMaths.o
```

and linking it, now telling the linker to link it to the “math” library:

```
$ gcc myMaths.o -lm -o myMaths
```

[link (-l) to a library called libm.so or libm.a]

Now run it:

```
$ myMaths
cos(1.234000) = 0.330465
```

Summary

- C program structure
  - Separate code modules
    - Header files with declarations
    - Implementations in .c files
  - Compiling and linking
    - Object code in .o files can be re-used separately from original source code
  - Storage classes
    - Static for private functions
    - Extern for globally accessible functions
  - Libraries
    - Collections of object files

Makefiles: semi-automatic compilation

Dependencies:

```
myProgram.o:  myProgram.c
              stack.h

stack.o:      stack.c
              stack.h

myProgram:    myProgram.o
              stack.o
```

stack.h
- #include's
- stack structure definition
- stack function prototypes

myProgram.c
- **#include "stack.h"**
- main function
- other function definitions

stack.c
- **#include "stack.h"**
- stack function definitions

If anything `myProgram` depends upon changes, the affected components should be recompiled.

Makefiles: semi-automatic compilation

University of Exeter

Makefile: **Targets, dependencies and commands**

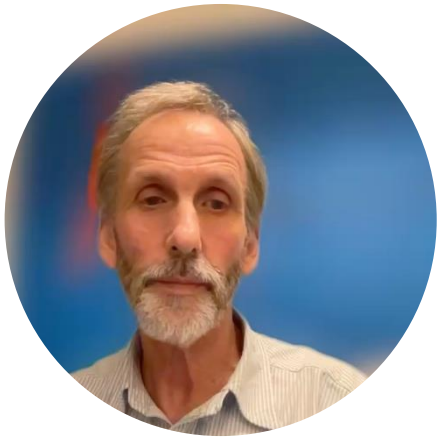
```
CC = gcc                # Use the gcc compiler
CFLAGS = -std=c99       # Flags for C compilation

stack.o: stack.c stack.h
<tab>   $(CC) $(CFLAGS) -c stack.c

myProgram.o: myProgram.c stack.h
<tab>   $(CC) $(CFLAGS) -c myProgram.c


myProgram: myProgram.o stack.o
<tab>   $(CC) $(CFLAGS)  myProgram.o stack.o -o myProgram
```

make

```
$ ls
Makefile  Makefile-simple  myProgram.c stack.c  stack.h
$ make myProgram
gcc –std=c99 -c myProgram.c
gcc –std=c99 -c stack.c
gcc –std=c99  myProgram.o stack.o -o myProgram
```

make

```
$ ls
Makefile  Makefile-simple  myProgram.c stack.c  stack.h
$ make myProgram
gcc –std=c99 -c myProgram.c
gcc –std=c99 -c stack.c
gcc –std=c99  myProgram.o stack.o -o myProgram
$ make myProgram
make: 'myProgram' is up to date
```



make

```
$ ls
Makefile  Makefile-simple  myProgram.c stack.c  stack.h
$ make myProgram
gcc –std=c99 -c myProgram.c
gcc –std=c99 -c stack.c
gcc –std=c99  myProgram.o stack.o -o myProgram
$ make myProgram
make: 'myProgram' is up to date
$ touch stack.c                # Update stack.c's modification time
$ make myProgram
gcc –std=c99 -c stack.c
gcc –std=c99 myProgram.o stack.o -o myProgram
```

Makefiles: semi-automatic compilation

Simplified Makefile:

make knows many standard rules: e.g. .o from .c

```
CC = gcc                # Use the gcc compiler
CFLAGS = -std=c99       # Flags for C compilation

myProgram: myProgram.o stack.o        # First, default target

stack.o: stack.c stack.h

myProgram.o: myProgram.c stack.h


clean:
        rm -f *.o myProgram
```

Summary

- C program structure
  - Separate code modules
  - Compiling and linking
  - Storage classes
  - Libraries

- **`make`**
  - Specify dependencies and rules
  - The only way to manage large projects: cmake for C++
  - https://www.gnu.org/software/make/